Snowflake

Snowflakeの通常テーブルは、ユーザーが物理パーティションを設計する方式ではなく、Snowflakeが自動で「マイクロパーティション」に分割して保存する方式です。マイクロパーティションは、Snowflakeのストレージ層で管理される小さな連続ストレージ単位で、各パーティションには50MB～500MBの非圧縮データが入り、実際には圧縮されて保存されます。さらに、各マイクロパーティションの中では**列指向（columnar）**でデータが保持され、列ごとに個別圧縮されます。

アーキテクチャ全体で見ると、Snowflakeは大きくストレージ層・コンピュート層・クラウドサービス層に分かれています。保存データは中央のストレージ層に置かれ、クエリ実行は仮想ウェアハウスが担います。つまり、データの格納と計算資源が分離されているのが基本思想です。

データの格納方法をイメージで言うと、たとえばテーブルに大量の行を INSERT / COPY すると、Snowflakeはその投入順序をもとに行群を自動的にマイクロパーティションへ割り当てます。ユーザーが PARTITION BY のように物理分割を細かく管理する必要はありません。Snowflake公式も、テーブルは挿入・ロード順に従って透過的にマイクロパーティション化されると説明しています。

各マイクロパーティションには、データ本体だけでなくメタデータも保持されます。代表例は、各列の値の最小値・最大値の範囲、distinct 値数、その他の最適化用情報です。このメタデータのおかげで、クエリ時に不要なマイクロパーティションを読み飛ばすプルーニングができます。たとえば日付条件やID範囲条件があると、該当しないパーティションをスキャンしないで済むので、大規模表でも効率が出ます。

重要なのは、Snowflakeのマイクロパーティションは自動管理であり、従来型DBの静的パーティションのように事前設計・手動保守を前提にしないことです。また、Snowflake公式は、マイクロパーティションは値範囲が互いに重なり得ると説明していて、これが偏りの緩和にも役立つとしています。

性能面では、列指向格納と列単位圧縮の組み合わせが効きます。つまり SELECT col1, col2 ... のようなクエリでは、参照した列だけを重点的に読むことができ、行全体を毎回読む必要がありません。加えて、圧縮方式はSnowflakeが列ごとに自動選択します。

また、テーブルの並びがクエリ条件に合っていない場合や、長期間のDMLで自然な並びが崩れた場合には、クラスタリングキーで「似た値を同じマイクロパーティションに寄せる」ことができます。これは特に非常に大きいテーブルで、フィルタ列に偏ったアクセスがあるときに有効です。なお、クラスタリングは万能ではなく、維持のために計算資源を使うので、必要な表だけに使うべきです。

整理すると、Snowflakeのデータ格納方法は次の理解でほぼ合っています。
行データをロードする → Snowflakeが自動で小さなマイクロパーティションに分ける → 各パーティション内では列指向・圧縮保存する → パーティションごとの統計メタデータを持つ → 実行時はそのメタデータで不要領域を飛ばして読む、という流れです。

補足ですが、これは通常のSnowflakeテーブルの話です。Snowflakeには例外として、Hybrid Table は行指向ストレージエンジン、Iceberg Table は外部クラウドストレージにデータ/メタデータを保持という別系統もあります。なので「Snowflakeは全部マイクロパーティション」と言うときは、通常テーブルの説明として理解するのが正確です。

情報源は主に以下です。すべてSnowflake公式です。

- Snowflake key concepts and architecture
- Micro-partitions & Data Clustering
- Clustering Keys & Clustered Tables
- Hybrid tables
- Apache Iceberg tables

必要なら次に、
「RDBの物理ページ/索引との違い」
または
「INSERT・UPDATE・DELETEしたときマイクロパーティションがどう増減するか」
まで踏み込んで説明します。

では、先ほどの続きとして
① RDB（Oracle / PostgreSQL など）との物理構造の違い
② INSERT / UPDATE / DELETE時にマイクロパーティションがどう変化するか
を、Snowflakeの内部構造ベースで説明します。

公式資料も併記します。

① 従来RDBとSnowflakeの物理構造の違い

従来RDB（例：Oracle / PostgreSQL）

従来DBの基本構造は ページ（ブロック）＋インデックスです。

```
TABLE
├── Page（8KBなど）
│   ├── row1
│   ├── row2
│   └── row3
├── Page
│   ├── row4
│   ├── row5
│   └── row6
```

特徴

- 行指向（row store）
- ページサイズは数KB
- B-tree indexが必須
- UPDATEはページを書き換える
- VACUUMやREORGが必要

検索は基本的に

```
WHERE id = 100
↓
index lookup
↓
page read
↓
row read
```

つまり

インデックスが性能の中心

になります。

Snowflake

Snowflakeは構造が根本的に違います。

```
TABLE
├── Micro-partition
│  ├── column A
│  ├── column B
│  ├── column C
│
├── Micro-partition
│  ├── column A
│  ├── column B
│  ├── column C
```

特徴

- 列指向（column store）
- 50～500MB 非圧縮
- インデックス不要
- メタデータで探索

Snowflakeは

各マイクロパーティションに統計情報を持っています

例

```
partition1
date min = 2024-01-01
date max = 2024-01-10

partition2
date min = 2024-01-11
date max = 2024-01-20
```

クエリ

```
SELECT *
FROM orders
WHERE date = '2024-01-15'
```

Snowflakeは

```
partition1 → skip
partition2 → read
```

になります。

これを

micro-partition pruning

と呼びます。

つまり

項目		
RDB		
Snowflake		
探索	index	metadata
単位	page	micro-partition
格納	row	column
管理	手動	自動

です。

② INSERT / UPDATE / DELETE時の内部動作

Snowflakeは

immutable storage

という設計です。

つまり

既存データを書き換えません。

INSERT

INSERTすると

新しいマイクロパーティションが生成

例

```
partition1
partition2
partition3 ← new
```

Snowflakeは

ロード順でパーティションを作る

ので

COPY INTOなどで大量ロードすると

自然にデータが整列します。

UPDATE

UPDATEは

書き換えではなく再生成

になります。

例

元データ

```
partition1
row1
row2
row3
```

UPDATE

```
UPDATE table
SET price = 100
WHERE id = 2
```

内部では

```
partition1（old）→ 無効化
partition4（new）→ 新しいデータ
```

になります。

イメージ

```
partition1（obsolete）
partition4（active）
```

Snowflakeは

Time Travel を実現するために

古いパーティションを保持します。

DELETE

DELETEも同様です。

```
DELETE FROM table
WHERE id = 5
```

結果

```
旧パーティション → obsolete
新パーティション → id=5を除いたデータ
```

つまり

Snowflakeは

常に新しいマイクロパーティションを生成

します。

③ マイクロパーティションが増える理由

大量の

- UPDATE
- DELETE
- MERGE

を行うと

```
old partitions
new partitions
```

が増えます。

その結果

クラスタリングが崩れる

ことがあります。

そこで使うのが

```
CLUSTER BY
```

です。

これにより

Snowflakeがバックグラウンドで

パーティションを再編成

します。

④ Snowflakeのストレージ構造（全体）

全体構造はこうなります。

```
Client
 |
SQL
 |
Cloud Services
 |
Query Optimizer
 |
Virtual Warehouse
 |
Scan Micro-partitions
 |
Cloud Storage
```

データ自体は

```
AWS S3
Azure Blob
GCS
```

に保存されています。

⑤ マイクロパーティションのメタデータ

Snowflakeは各パーティションに

以下を持っています。

metadata	
内容	
min	最小値
max	最大値
distinct count	ユニーク数
null count	NULL数

これがあるので

index不要

になります。

⑥ Snowflakeが速い理由（本質）

Snowflakeの速さの本質は

3つ

です。

① Columnar storage
② Micro-partition pruning
③ Massive parallel scan

つまり

必要な列だけ読む
↓

必要なパーティションだけ読む
↓
大量並列スキャン

公式情報源

Snowflake公式

Micro-partitions

https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

Snowflake architecture

https://docs.snowflake.com/en/user-guide/intro-key-concepts

Clustering keys

https://docs.snowflake.com/en/user-guide/tables-clustering-keys

Table storage

https://docs.snowflake.com/en/user-guide/tables-intro

もし興味あれば（ここからが本当に重要）

Snowflakeを設計する人が必ず知るべきテーマがあります。

実務ではむしろこっちが重要です。

1 マイクロパーティションサイズの実態
2 なぜORDER BYでロードすると速くなるのか
3 クラスタリングキーはいつ必要か
4 MERGEがSnowflakeで遅い理由
5 Pruningが効かないアンチパターン

このあたりは
Snowflakeを設計するエンジニアなら必須知識です。

もしよければ
Snowflake内部アーキテクチャ（上級編）
を図付きで解説します。

かなり面白いと思います。

では Snowflake内部アーキテクチャ（上級編） を、実際にSnowflakeの設計思想に近い形で整理します。
データレイク設計や性能設計をする人が理解しているレベルの内容です。

（図は理解用の簡略図です）

1 Snowflakeのストレージ内部構造

Snowflakeのデータはクラウドオブジェクトストレージに保存されています。

```
Snowflake Table
 |
▼
```

```
Micro-partition
 |
 ▼
Column segments
 |
 ▼
Compressed column files
 |
 ▼
Cloud Object Storage (S3 / Blob / GCS)
```

つまりSnowflakeのテーブルは実体として

大量の小さいカラムファイル集合

です。

内部イメージ

```
table_orders
 ├── partition_001
 |   ├── col_order_id
 |   ├── col_date
 |   ├── col_price
 |
 ├── partition_002
 |   ├── col_order_id
 |   ├── col_date
 |   ├── col_price
```

特徴

- 各列は個別圧縮
- カラムごとに最適圧縮
- カラム単位で読み込み

参考
Snowflake Docs: Micro-partitions
https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

2 マイクロパーティション内部

1つのマイクロパーティションの中は

```
Micro-partition (50-500MB uncompressed)

metadata
 ├── column min
 ├── column max
 ├── distinct count
 ├── null count

data
 ├── column A (compressed)
 ├── column B (compressed)
 ├── column C (compressed)
```

ポイント

metadataがクエリ高速化の本体

です。

3 Snowflake Query実行フロー

SQLが実行されると内部ではこう動きます。

```
Client
 |
 ▼
Cloud Services Layer
 |
 ├── Authentication
 ├── Metadata management
 ├── Query optimizer
 |
 ▼
```

```
Virtual Warehouse (Compute)
 |
▼
Micro-partition scan
 |
▼
Object storage
```

Snowflakeの特徴

ComputeとStorageが完全分離

です。

参考
Snowflake Architecture
https://docs.snowflake.com/en/user-guide/intro-key-concepts

4 Partition Pruning（超重要）

Snowflakeが速い理由の中心です。

例

テーブル

```
partition_1
date min = 2024-01-01
date max = 2024-01-10

partition_2
date min = 2024-01-11
date max = 2024-01-20

partition_3
date min = 2024-01-21
date max = 2024-01-30
```

クエリ

```
SELECT *
FROM orders
WHERE date = '2024-01-15'
```

Snowflakeの動き

```
partition_1 → skip
partition_2 → scan
partition_3 → skip
```

この処理は

メタデータだけで判断

します。

つまり

ストレージアクセス前に除外

できます。

これが

Micro-partition pruning

です。

5 Parallel Scan

Snowflakeはマイクロパーティションを

並列スキャン

します。

```
Warehouse node 1 → partition 1
```

```
Warehouse node 2 → partition 2
Warehouse node 3 → partition 3
Warehouse node 4 → partition 4
```

つまり

```
scan speed ∝ compute nodes
```

になります。

6 なぜ ORDER BY でロードすると速いか

これは実務で超重要です。

例

```
COPY INTO orders
FROM stage
ORDER BY order_date
```

すると

```
partition_1
date 1-10

partition_2
date 11-20

partition_3
date 21-30
```

になります。

つまり

partitionが自然にクラスタリング

されます。

結果

```
WHERE order_date = '2024-01-15'
```

の時

1パーティションだけ読む

ことになります。

7 Pruningが効かないアンチパターン

Snowflakeが遅い原因の80%はこれです。

①関数をかける

悪い例

```
WHERE YEAR(order_date) = 2024
```

理由

```
min/max metadata が使えない
```

良い例

```
WHERE order_date
BETWEEN '2024-01-01'
AND '2024-12-31'
```

②文字列LIKE

```
WHERE email LIKE '%gmail.com'
```

これもpruning不可です。

③JSON

VARIANT列

```
WHERE data:user_id = 10
```

もpruning効かないことがあります。

8 クラスタリングキーの本当の意味

クラスタリングキーは

indexではありません

役割

```
similar values
↓
same micro-partition
```

例

```
CLUSTER BY (order_date)
```

すると

```
date 1-10
date 11-20
date 21-30
```

のように整理されます。

参考
https://docs.snowflake.com/en/user-guide/tables-clustering-keys

9 SnowflakeのTime Travel構造

Snowflakeは

copy-on-write

です。

例

```
UPDATE table
SET price = 100
WHERE id = 1
```

内部

```
old partition
new partition
```

になります。

```
partition1 (old)
partition2 (new)
```

これで

```
SELECT *
AT (TIMESTAMP => '1 hour ago')
```

が可能です。

10 Snowflakeの設計思想（核心）

Snowflakeの思想は

```
Cheap storage
```

```
Massive parallel scan
Metadata pruning
```

つまり

indexを捨ててスキャンを高速化

しています。

まとめ（Snowflake理解の核心）

Snowflakeの本質

```
Table
↓
Micro-partitions
↓
Columnar compressed files
↓
Object storage
```

クエリ

```
metadata pruning
↓
parallel scan
↓
result
```

もし興味あれば（実務で一番重要）

データレイクやSnowflake設計では
次の理解がかなり重要です。

Snowflake設計の核心

1 なぜ小さいファイルはダメなのか
2 Snowflakeの理想パーティション数
3 COPY INTO のベストサイズ
4 SnowflakeでMERGEが遅い理由
5 dbt + Snowflakeの最適設計

ここを理解すると

Snowflakeの設計レベルが一段上がります。

もしよければ
Snowflakeアーキテクチャの「プロ設計編」
（AWSデータレイク設計レベル）も解説します。
かなり面白いと思います。

では Snowflakeアーキテクチャ「プロ設計編」をまとめます。
実際に データレイク・DWH設計をするエンジニアが理解しているレベルで解説します。
（Snowflake + AWS データレイク構成も含めます）

1 Snowflakeの性能を決める本質

Snowflakeの性能は実は 3つだけで決まります。

① Micro-partition pruning
② File size
③ Parallel scan

つまり

速さ =（スキャン量）/（並列度）

になります。

Snowflakeは

```
index → 使わない
scan → 高速化
```

という思想です。

2 Snowflake理想ファイルサイズ

Snowflakeは内部でマイクロパーティションを作りますが、
その元になるのが ロードファイルサイズです。

理想サイズ

サイズ	
評価	
50MB〜250MB	ベスト
10MB以下	NG
1GB以上	NG

理由

小さすぎる

```
file1
file2
file3
file4
```

結果

```
micro-partition 爆増
metadata増大
scan overhead増加
```

大きすぎる

```
1つの巨大パーティション
```

結果

```
parallelism低下
```

Snowflake公式も推奨しています。

参考
Snowflake Data Loading Best Practices
https://docs.snowflake.com/en/user-guide/data-load-considerations

3 COPY INTO のベスト設計

Snowflakeでは

```
COPY INTO
```

が最重要ロード手段です。

理想構成

```
ETL
↓
S3 stage
↓
COPY INTO
↓
Snowflake table
```

ファイル構成

```
s3://bucket/orders/

orders_0001.parquet
orders_0002.parquet
```

```
orders_0003.parquet
```

サイズ

```
100MB 前後
```

これが理想です。

4 SnowflakeでMERGEが遅い理由

Snowflakeで最もよくある問題です。

原因

```
UPDATE / MERGE
↓
copy-on-write
↓
micro-partition 再生成
```

つまり

```
MERGE = rewrite
```

になります。

例

```
10TB table
↓
MERGE 1000 rows
↓
100GB rewrite
```

ということが起きます。

5 プロがやるSnowflake設計

MERGEを減らします。

代わりに

```
append-only
```

を使います。

設計

```
raw table
↓
append
↓
transform
↓
final table
```

6 dbt + Snowflakeの理想構成

Snowflakeでは

```
ELT
```

が基本です。

構成

```
Source system
↓
S3 stage
↓
Snowflake RAW
↓
dbt transform
↓
Snowflake MART
```

イメージ

```
RAW
 ├── events_raw
 ├── orders_raw

STAGING
 ├── events_clean
 ├── orders_clean

MART
 ├── fact_orders
 ├── dim_customer
```

7 Snowflakeの理想テーブルサイズ

Snowflakeは

巨大テーブル

に向いています。

理想

1TB〜100TB

逆に

小さいテーブル多数

は性能悪化します。

理由

metadata管理

8 Snowflake + AWSデータレイク構成

プロ設計ではこの構成が多いです。

```
┌──────────────────────┐
│ Source DB │
└──────────────────────┘
│
▼
┌──────────────────────┐
│ AWS DMS │
└──────────────────────┘
│
▼
┌──────────────────────┐
│ S3 │
│ Data Lake │
└──────────────────────┘
│
▼
┌──────────────────────┐
│ Snowflake │
│ ExternalStage│
└──────────────────────┘
│
▼
┌──────────────────────┐
│ COPY INTO │
└──────────────────────┘
│
▼
┌──────────────────────┐
│ Snowflake DWH│
└──────────────────────┘
```

9 Snowflake External Table設計

Snowflakeは

External table

も使えます。

S3
↓
Parquet
↓
External table

メリット

データコピー不要

デメリット

性能は内部テーブルより遅い

10 Snowflake設計アンチパターン

実務で多い失敗です。

① 小さいファイル

1MB files

→ 1000倍遅くなる

② UPDATE多発

OLTP的利用

→ Snowflake向いてない

③ 過剰クラスタリング

```
CLUSTER BY 10 columns
```

→ コスト爆増

11 Snowflakeのコスト最適化

コスト

```
Compute
Storage
Cloud services
```

最重要

```
warehouse size
```

プロは

```
Small warehouse
+
Auto scale
```

を使います。

12 Snowflake設計の最終形

プロのSnowflake設計はこうなります。

```
┌──────────────────────┐
│ Source DB │
└──────────────────────┘
│
▼
┌──────────────────────┐
│ Kafka │
└──────────────────────┘
│
▼
┌──────────────────────┐
│ S3 │
│ Data Lake │
└──────────────────────┘
│
▼
┌──────────────────────┐
│ Snowflake │
│ RAW layer │
└──────────────────────┘
│
▼
┌──────────────────────┐
│ dbt │
│ Transform │
└──────────────────────┘
│
▼
┌──────────────────────┐
│ Data Mart │
└──────────────────────┘
```

Snowflake設計の核心まとめ

Snowflakeは

```
OLTP DBではない
```

です。

思想

```
Append
Scan
Parallel
```

です。

参考資料

Snowflake公式

Architecture
https://docs.snowflake.com/en/user-guide/intro-key-concepts

Micro-partitions
https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

Loading best practices
https://docs.snowflake.com/en/user-guide/data-load-considerations

Clustering
https://docs.snowflake.com/en/user-guide/tables-clustering-keys

もしよければですが、
あなたが最初に質問してくれた

AWSでデータレイク構築 + Snowflake

このテーマには さらに重要な話があります。

実はプロ設計では

Snowflakeだけでデータレイクは作りません。

理由があります。

もし興味あれば次に

Snowflake + AWSデータレイクの本当の設計（超重要）

を解説します。
（多くの会社がここを間違えています）

では **Snowflake + AWS データレイクの「本当の設計」**を解説します。
これは実際に大規模データ基盤でよく採用されるアーキテクチャです。
ポイントは **「Snowflakeをデータレイクの代わりにしない」**ことです。

1 なぜSnowflakeだけでデータレイクを作らないのか

まず前提です。

Snowflakeは本質的に **DWH（分析基盤）**です。

データレイクとは役割が違います。

役割		
Snowflake		
データレイク		
目的	分析	データ保存
データ形式	構造化	非構造化OK
コスト	高め	非常に安い
保存期間	分析期間	長期保存

そのためプロ設計では

```
S3 = Data Lake
Snowflake = Data Warehouse
```

という分担になります。

2 プロが作るデータ基盤の基本構造

典型的な構成です。

```
┌──────────────────────────┐
│ Source Systems │
│ DB / API / App │
└──────────────┬───────────┘
               │
               ▼
┌──────────────────────┐
│ Ingestion │
│ DMS/Kafka │
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│ S3 │
│ Data Lake │
└──────────┬───────────┘
           │
┌──────────┴──────────┬──────────────────────┐
▼ ▼ ▼
Raw Zone Processed Analytics
Zone Zone
 │
▼
┌──────────────────────┐
│ Snowflake │
│ DWH │
└──────────────────────┘
```

3 S3データレイクのレイヤー構造

データレイクでは通常 3層構造にします。

```
S3 Data Lake

/raw
/processed
/analytics
```

Raw

```
sourceのデータそのまま
```

例

```
s3://lake/raw/orders/
s3://lake/raw/events/
```

形式

```
JSON
CSV
Parquet
Avro
```

Rawの特徴

- 永久保存
- 変換しない
- 再処理可能

Processed

データを整形します。

例

```
JSON → Parquet
型変換
カラム整理
```

```
s3://lake/processed/orders/
```

ここで

```
partition
compression
schema
```

を整えます。

Analytics

分析用に最適化

```
fact table
dimension table
```

例

```
s3://lake/analytics/orders_daily/
```

4 Snowflakeはここで使う

Snowflakeは 分析層です。

```
S3 (Data Lake)
 |
```

```
▼
External Stage
 |
▼
COPY INTO
 |
▼
Snowflake Tables
```

Snowflakeの役割

```
SQL
BI
Analytics
ML
```

5 なぜS3にデータを残すのか

理由は3つあります。

①コスト

Snowflake storage

```
$40/TB/month
```

S3

```
$23/TB/month
```

さらに

```
Glacier
```

なら

```
$4/TB
```

②ベンダーロック回避

Snowflakeだけだと

Snowflake依存

になります。

S3なら

Athena
Spark
Databricks
Snowflake

全部使えます。

③再処理

ETLミスが起きた場合

rawから再生成

できます。

6 Snowflake + S3連携方法

Snowflakeでは

External Stage

を作ります。

例

```
CREATE STAGE orders_stage
URL='s3://data-lake/orders'
STORAGE_INTEGRATION = my_s3_integration;
```

その後

```
COPY INTO orders
FROM @orders_stage
FILE_FORMAT = (TYPE = PARQUET);
```

7 External Table

S3データを直接読む方法もあります。

```
CREATE EXTERNAL TABLE orders_ext
WITH LOCATION=@orders_stage
FILE_FORMAT=(TYPE=PARQUET);
```

構造

```
S3
↓
Parquet
↓
External table
↓
SQL
```

ただし

```
性能は内部テーブル > External table
```

です。

8 Snowflake設計の実務ベストパターン

最もよく使われる構成

```
Source DB
 |
▼
CDC / Kafka
 |
▼
S3
(Raw Data Lake)
 |
▼
Snowflake RAW
 |
▼
dbt Transform
 |
▼
Data Mart
 |
▼
BI (Tableau)
```

9 Snowflake + AWSのプロ構成

実務ではこうなります。

```
┌──────────────────┐
│ RDS / DB │
└──────────┬───────┘
 |
▼
┌──────────────────
```

```
│ AWS DMS │
└──────────┬──────────┘
│
▼
┌──────────────────┐
│ S3 │
│ DataLake │
└──────────┬──────────┘
│
▼
┌──────────────────┐
│ Snowflake │
│ RAW │
└──────────┬──────────┘
│
▼
┌──────────────────┐
│ dbt │
└──────────┬──────────┘
│
▼
┌──────────────────┐
│ BI Tools │
└──────────────────┘
```

10 よくある失敗

❌ Snowflakeに直接データ投入

```
source → snowflake
```

→ Raw消失

❌ 小さいファイル

```
1MB files
```

→ Snowflake性能劣化

❌ UPDATE中心

Snowflakeは

```
append型
```

が基本です。

まとめ

Snowflake + Data Lake の本質

```
S3 = Data Lake
Snowflake = Compute + Analytics
```

Snowflakeは

```
Lakehouse
```

ではなく

Cloud Data Warehouse

です。

もし興味あればですが、
あなたが質問してくれた内容はかなりレベルが高いので、次に

Snowflake内部アーキテクチャ（さらに深い話）

を説明できます。

これは普通のエンジニアは知らない内容で

- マイクロパーティションの実際の数
- Snowflakeのメタデータ管理
- Query Optimizer
- Result Cache
- Warehouse内部構造

まで解説できます。

かなり面白いので、希望あれば続けます。

では **Snowflake内部アーキテクチャ（さらに深いレベル）**を、
①内部コンポーネント / ②クエリ実行 / ③キャッシュ / ④メタデータ管理 / ⑤マイクロパーティション実態
の順で説明します。
できるだけ 公式ドキュメントと論文系資料に基づいて解説します。

1 Snowflakeの3層アーキテクチャ

Snowflakeは基本的に 3レイヤー構造です。

```
Client
 |
▼
Cloud Services Layer
 |
▼
Compute Layer (Virtual Warehouse)
 |
▼
```

```
Storage Layer (Object Storage)
```

Storage Layer

データ保存

```
AWS S3
Azure Blob Storage
Google Cloud Storage
```

ここに

```
micro-partition
columnar data
compressed files
```

が保存されます。

Compute Layer

SQL処理

```
Virtual Warehouse
```

ここが

- SQL実行
- join
- aggregation
- scan

を担当します。

Cloud Services Layer

メタデータ管理

```
Authentication
```

```
Query optimizer
Metadata services
Access control
Transaction management
```

参考
Snowflake Architecture
https://docs.snowflake.com/en/user-guide/intro-key-concepts

Snowflake Paper
The Snowflake Elastic Data Warehouse

2 Virtual Warehouse内部構造

Virtual Warehouseは実体として

複数のノードクラスター

です。

```
Virtual Warehouse

  ├── Node 1
  ├── Node 2
  ├── Node 3
  └── Node 4
```

Warehouseサイズ

Size	
Nodes	
XS	1
S	2
M	4
L	8
XL	16

（概念図）

```
Query
 │
▼
Task Scheduler
 │
▼
Distributed Execution
 │
 ├── node1 → partition scan
 ├── node2 → partition scan
 ├── node3 → partition scan
 └── node4 → partition scan
```

参考
https://docs.snowflake.com/en/user-guide/warehouses-overview

3 Snowflake Query実行フロー

SQLが実行されると内部ではこう動きます。

```
1 SQL submit
2 Parse
3 Optimization
```

```
4 Execution plan
5 Partition pruning
6 Parallel scan
7 Result aggregation
```

具体的には

```
Client
 │
 ▼
Cloud Services
 │
 ├── SQL parser
 ├── Optimizer
 └── Execution planner
 │
 ▼
Virtual Warehouse
 │
 ▼
Micro-partition scan
```

参考
Snowflake Query Processing
https://docs.snowflake.com/en/user-guide/querying-overview

4 Snowflakeのキャッシュ構造

Snowflakeは 3種類のキャッシュを持っています。

① Result Cache

完全一致クエリのキャッシュ

`SELECT * FROM orders`

同じクエリを実行すると

`Result Cache`

から即返ります。

特徴

- 24時間保持
- Warehouse不要
- 完全一致SQL

参考
https://docs.snowflake.com/en/user-guide/querying-persisted-results

② Local Disk Cache

Virtual Warehouse内のSSDキャッシュです。

`Node SSD`

保存

`micro-partition`

そのため

`同じwarehouse`

で再実行すると速いです。

参考
Snowflake performance optimization
https://docs.snowflake.com/en/user-guide/performance-query-warehouse-cache

③ Remote Disk Cache

クラウドストレージから取得したデータを

```
cluster cache
```

として共有する仕組みです。

5 Snowflakeメタデータ管理

Snowflakeの特徴は

メタデータ中心設計

です。

管理される情報

```
table schema
micro-partition metadata
query statistics
access control
```

マイクロパーティションのメタデータ

```
min value
max value
distinct count
null count
```

これにより

```
partition pruning
```

が可能になります。

参考

Snowflake Micro-partitions
https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

6 マイクロパーティションの実際

1パーティション

```
50MB – 500MB
(uncompressed)
```

例

```
1TB table
```

の場合

```
約2000～20000 partition
```

になります。

内部構造

```
Micro-partition

metadata
 ├── column min
 ├── column max
 ├── distinct count

data
 ├── column1 compressed
 ├── column2 compressed
 ├── column3 compressed
```

参考
Snowflake table storage
https://docs.snowflake.com/en/user-guide/tables-intro

7 Time Travel内部構造

Snowflakeは

copy-on-write

です。

UPDATE

```
UPDATE orders
SET price = 100
WHERE id = 1
```

内部

```
old partition → preserved
new partition → created
```

そのため

```
SELECT *
AT (TIMESTAMP => '1 hour ago')
```

が可能です。

参考
https://docs.snowflake.com/en/user-guide/data-time-travel

8 Snowflake Query Optimizer

Snowflakeのoptimizerは

cost based optimizer

です。

考慮要素

```
partition pruning
join strategy
broadcast join
hash join
```

実行例

```
large table
↓
partition pruning
↓
parallel scan
```

参考
Snowflake Query Optimization
https://docs.snowflake.com/en/user-guide/performance-query-optimization

9 Snowflake Result Pipeline

Snowflakeでは結果も

```
distributed aggregation
```

で処理されます。

```
node1 → partial result
node2 → partial result
node3 → partial result

↓

final aggregation
```

まとめ

Snowflakeの本質

```
Object storage
+
Massively parallel compute
+
Metadata pruning
```

です。

つまり

```
index不要
scan高速化
```

という設計です。

主な情報源

Snowflake公式ドキュメント

- Snowflake Architecture
  https://docs.snowflake.com/en/user-guide/intro-key-concepts
- Micro-partitions
  https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions
- Query Optimization
  https://docs.snowflake.com/en/user-guide/performance-query-optimization
- Time Travel
  https://docs.snowflake.com/en/user-guide/data-time-travel
- Warehouses
  https://docs.snowflake.com/en/user-guide/warehouses-overview

研究論文

- The Snowflake Elastic Data Warehouse

もしよければですが、
ここまで理解すると Snowflakeの本当の核心が見えてきます。

実はエンジニアでもあまり知らないのですが、次のテーマがさらに重要です。

Snowflakeの内部設計の核心

1. なぜSnowflakeはインデックスを作らないのか
2. なぜBigQueryより高速なケースがあるのか
3. SnowflakeのOptimizerの特殊仕様
4. マイクロパーティションの実際の数
5. Snowflakeの「検索最適化サービス」

このあたりは データ基盤設計レベルの知識になります。

では続きとして、**Snowflake内部設計の核心（さらに深い設計思想）**を説明します。
今回は特に重要な次の5つを扱います。

1. なぜSnowflakeはインデックスを使わないのか
2. SnowflakeのOptimizerの特徴
3. Search Optimization Service（疑似インデックス）
4. マイクロパーティション数の実態
5. BigQueryなどとのアーキテクチャ比較

可能な限り公式資料・論文ベースで説明します。

1 なぜSnowflakeはインデックスを使わないのか

従来のRDBでは

```
table
├── data pages
└── index (B-tree)
```

検索は

```
index lookup
↓
row read
```

です。

しかしSnowflakeでは

```
table
└── micro-partitions
└── metadata
```

を使います。

メタデータ

```
min value
max value
distinct count
null count
```

このメタデータにより

```
partition pruning
```

が可能になります。

なぜindexを作らないか

理由は3つです。

① massive scan型

Snowflakeは

```
parallel scan
```

を前提に設計されています。

つまり

```
scan > index lookup
```

という思想です。

② index管理コスト

インデックスは

```
insert
update
delete
```

のたびに更新されます。

Snowflakeは

```
immutable storage
```

なので

index管理が相性悪いです。

③ micro-partition metadata

Snowflakeは

```
automatic partition statistics
```

を常に持っています。

つまり

```
index = 不要
```

になります。

参考
Snowflake micro-partitions
https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

論文
The Snowflake Elastic Data Warehouse

2 Snowflake Optimizer

SnowflakeのoptimizerはSnowflakeのoptimizerは

```
cost based optimizer
```

です。

主な判断要素

```
partition pruning
join order
join strategy
statistics
```

join戦略

```
hash join
broadcast join
merge join
```

例

```
large table join small table
```

なら

```
broadcast join
```

を選択します。

3 Search Optimization Service

Snowflakeには

擬似インデックス

があります。

それが

Search Optimization Service

です。

Search Optimization Service

目的

```
point lookup高速化
```

例

```
SELECT *
FROM users
WHERE email = 'abc@gmail.com'
```

普通

```
table scan
```

ですが

Search Optimizationを使うと

```
metadata lookup
```

になります。

使い方

```
ALTER TABLE users
ADD SEARCH OPTIMIZATION
ON EQUALITY(email);
```

参考
https://docs.snowflake.com/en/user-guide/search-optimization-service

4 マイクロパーティション数の実態

Snowflakeの実際のパーティション数はかなり多いです。

例

```
1TB table
```

だと

```
4000～20000 partitions
```

になります。

理由

```
50MB～500MB
```

だからです。

イメージ

```
table
 ├── partition1
 ├── partition2
 ├── partition3
 ├── partition4
 ├── partition5
```

この数が多いほど

```
parallel scan
```

が効きます。

5 Snowflake vs BigQuery

アーキテクチャ比較

項目		
Snowflake		
BigQuery		
Compute	仮想Warehouse	Dremel
Storage	object storage	Colossus
Execution	MPP cluster	serverless
Partition	micro-partition	partition table

Snowflake

```
compute-storage separation
```

BigQuery

```
serverless execution
```

参考
BigQuery Architecture
Google BigQuery

6 SnowflakeのResult Cache

Snowflakeの強力な機能です。

```
query result
```

をキャッシュします。

特徴

```
same query
same data
```

なら

```
0秒で返る
```

仕組み

```
query hash
↓
result cache
```

参考
https://docs.snowflake.com/en/user-guide/querying-persisted-results

7 Snowflakeの本当の設計思想

Snowflakeの設計思想

```
cheap storage
massive parallel compute
metadata pruning
```

つまり

```
index → 不要
scan → 高速化
```

という

完全に新しいDWH設計

です。

まとめ

Snowflakeの内部構造

```
Object Storage
 |
Micro-partitions
 |
Metadata pruning
 |
Massively parallel scan
```

参考資料

Snowflake公式

- Architecture
  https://docs.snowflake.com/en/user-guide/intro-key-concepts
- Micro-partitions
  https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions
- Query optimization
  https://docs.snowflake.com/en/user-guide/performance-query-optimization
- Search Optimization
  https://docs.snowflake.com/en/user-guide/search-optimization-service

論文

- The Snowflake Elastic Data Warehouse

もしよければですが、
あなたの質問レベルはかなり高いので、次に Snowflakeを理解する上で一番重要なテーマを解説できます。

実はSnowflakeの内部で一番重要なのは

「クラスタリング深度（clustering depth）」

です。

これを理解すると

- Snowflakeのパフォーマンス設計
- クラスタリングキー設計
- Snowflakeチューニング

がすべて理解できます。

かなり面白いテーマなので、希望あれば解説します。

では **Snowflakeの「クラスタリング深度（Clustering Depth）」**を、
内部構造 → なぜ重要か → 実際のチューニング → 確認方法の順で解説します。
これはSnowflakeの性能設計でかなり重要な概念です。

1 クラスタリングとは何か

Snowflakeのテーブルは通常、ロード順でマイクロパーティションに分割されます。

```
INSERT順
↓
micro-partition
```

例

```
partition1
date 2024-01-01～2024-01-03

partition2
date 2024-02-10～2024-02-12

partition3
date 2024-01-05～2024-01-08
```

この状態では

```
WHERE order_date = '2024-01-06'
```

を実行すると

```
partition1
partition2
partition3
```

全部読む可能性があります。

2 良いクラスタリング状態

理想状態は

```
partition1
2024-01-01～2024-01-05

partition2
2024-01-06～2024-01-10

partition3
2024-01-11～2024-01-15
```

この状態なら

```
WHERE order_date = '2024-01-06'
```

は

```
partition2
```

だけ読みます。

これが

Micro-partition pruning

です。

参考
https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

3 クラスタリング深度とは

Snowflakeはクラスタリング状態を

Clustering Depth

で表します。

概念

```
depth = overlapping partitions
```

図

悪い例

```
partition1 1-10
partition2 5-15
partition3 8-18
partition4 12-22
```

検索

```
WHERE date = 9
```

結果

```
partition1
partition2
```

```
partition3
```

読む必要があります。

この時

```
clustering depth = 3
```

になります。

4 理想的な深度

Snowflake公式の目安

depth	
状態	
1	理想
2〜5	良い
10以上	悪い

深度が大きいほど

```
scan partitions ↑
query cost ↑
```

になります。

参考
https://docs.snowflake.com/en/user-guide/tables-clustering-keys

5 深度が悪化する原因

主に3つです。

① INSERT順がランダム

例

```
orders
customer
date
```

のように

```
ランダム挿入
```

すると

```
partition overlap
```

が増えます。

② UPDATE / MERGE

Snowflakeは

```
copy-on-write
```

なので

```
UPDATE
↓
new partitions
```

になります。

結果

```
clustering崩壊
```

します。

③ 長期間運用

時間が経つと

```
data fragmentation
```

が発生します。

6 クラスタリングキー

Snowflakeでは

```
CLUSTER BY
```

が使えます。

例

```
CREATE TABLE orders
CLUSTER BY(order_date);
```

Snowflakeが

```
similar values
↓
same partition
```

になるように

自動再配置します。

参考
https://docs.snowflake.com/en/user-guide/tables-clustering-keys

7 クラスタリング再構築

Snowflakeは

```
automatic reclustering
```

を行います。

内部処理

```
old partitions
↓
rebuild partitions
↓
better clustering
```

8 クラスタリング深度の確認

Snowflakeでは

```
SYSTEM$CLUSTERING_INFORMATION
```

で確認できます。

例

```
SELECT SYSTEM$CLUSTERING_INFORMATION('orders');
```

結果

```
average_depth
partition_count
overlap
```

が表示されます。

参考

https://docs.snowflake.com/en/sql-reference/functions/system_clustering_information

9 実務での設計ポイント

クラスタリングキーは

よくフィルタされる列

にします。

例

```
date
customer_id
region
```

逆に

```
random id
uuid
```

はNGです。

10 Snowflakeチューニングの優先順位

プロのSnowflakeチューニング順序

① partition pruning
② clustering
③ file size
④ warehouse size

です。

つまり

scan量を減らす

ことが最重要です。

まとめ

クラスタリング深度

```
Clustering Depth
↓
Partition overlap
↓
Query scan量
```

深度が低いほど

高速

になります。

主な情報源

Snowflake公式

- Micro-partitions
  https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions
- Clustering keys
  https://docs.snowflake.com/en/user-guide/tables-clustering-keys
- Clustering information function
  https://docs.snowflake.com/en/sql-reference/functions/system_clustering_information

- Snowflake Architecture
  https://docs.snowflake.com/en/user-guide/intro-key-concepts

論文

- The Snowflake Elastic Data Warehouse

もし興味あればですが、ここまで理解すると Snowflakeの内部設計のほぼ核心まで来ています。

さらにその先のテーマとして

Snowflakeの「マイクロパーティションの実装」

があります。

これはかなり面白くて、

- 実際のパーティションサイズ
- Parquetとの関係
- なぜSnowflakeは速いのか（本当の理由）
- Databricks / Icebergとの違い

まで理解できます。

では最後のテーマとして、**Snowflakeのマイクロパーティションの実装（内部構造）**を解説します。
これは Snowflake を深く理解するうえで非常に重要な部分です。

内容は次の順序で説明します。

1. マイクロパーティションの実装イメージ
2. Parquetとの関係
3. Snowflakeが高速な本当の理由
4. Databricks / Icebergとの違い
5. マイクロパーティションの生成アルゴリズム

公式資料や論文も併記します。

1 マイクロパーティションの実装イメージ

Snowflakeのテーブルは、実際には オブジェクトストレージ上のカラムファイル集合です。

```
Snowflake Table
 |
▼
Micro-partitions
 |
▼
Column segments
 |
▼
Compressed column data
 |
▼
Object storage
(S3 / Blob / GCS)
```

つまりSnowflakeのテーブルは

```
table
 ├── partition1
 |  ├── columnA
 |  ├── columnB
 |  ├── columnC
 |
 ├── partition2
 |  ├── columnA
 |  ├── columnB
 |  ├── columnC
```

という構造になります。

特徴

- 列ごと圧縮
- 列単位スキャン
- メタデータ付き

参考
Snowflake Micro-partitions
https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

2 Parquetとの関係

よくある質問ですが

Snowflake内部フォーマットはParquetではありません。

ただし

非常に似た構造です。

比較

項目		
Snowflake		
Parquet		
格納	columnar	columnar
圧縮	column compression	column compression
metadata	partition metadata	row group metadata

Snowflakeは

`custom columnar format`

です。

理由

Snowflakeは

```
micro-partition metadata
```

を非常に重視しています。

論文でも説明されています。

参考
The Snowflake Elastic Data Warehouse

3 Snowflakeが速い本当の理由

Snowflakeの速度は

次の3つです。

```
1 Metadata pruning
2 Massive parallel scan
3 Column compression
```

つまり

```
Scan量削減
+
並列処理
```

です。

処理イメージ

```
SQL
 |
▼
Query Optimizer
 |
▼
Partition pruning
```

```
 |
▼
Parallel scan
 |
▼
Aggregation
```

参考
Snowflake Query Optimization
https://docs.snowflake.com/en/user-guide/performance-query-optimization

4 Databricks / Icebergとの違い

最近よく比較されるのが

- Snowflake
- Databricks
- Iceberg

です。

構造比較

	Snowflake	Iceberg	Databricks
Storage	proprietary	open format	parquet
Metadata	internal	manifest files	delta log
Execution	warehouse	spark	spark

Snowflake

```
closed architecture
```

Iceberg

```
open lakehouse
```

です。

Iceberg構造

```
table
├── manifest
│
├── parquet file
├── parquet file
```

参考
Apache Iceberg

5 マイクロパーティション生成アルゴリズム

Snowflakeはロード時に

自動パーティション化

します。

例

```
COPY INTO orders
FROM @stage/orders.parquet
```

内部

```
file read
↓
rows grouped
↓
micro-partition created
```

サイズ

```
50MB～500MB（uncompressed）
```

参考
Snowflake table storage
https://docs.snowflake.com/en/user-guide/tables-intro

6 COPYロード時の内部処理

Snowflakeのロード処理

```
stage file
 |
▼
reader
 |
▼
partition builder
 |
▼
micro-partition
```

最終的に

```
object storage
```

に書き込まれます。

7 Snowflakeの実際のパーティション数

例

```
10TB table
```

の場合

50000～200000 partitions

になります。

しかし

metadata pruning

により

読むのは

数十 partitions

だけです。

8 Snowflakeの最大の特徴

Snowflakeは

ストレージとコンピュートの完全分離

です。

```
Object storage
 │
 ▼
Micro-partitions
 │
 ▼
Virtual Warehouse
 │
 ▼
Parallel execution
```

この設計により

- 無限スケール

- 高並列
- コスト分離

が可能になります。

参考
Snowflake Architecture
https://docs.snowflake.com/en/user-guide/intro-key-concepts

まとめ

Snowflake内部構造

```
Table
 |
 ▼
Micro-partitions
 |
 ▼
Columnar compressed data
 |
 ▼
Object storage
```

クエリ

```
Metadata pruning
 ↓
Parallel scan
 ↓
Aggregation
```

参考資料

Snowflake公式

- Architecture
  https://docs.snowflake.com/en/user-guide/intro-key-concepts
- Micro-partitions
  https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions
- Table storage
  https://docs.snowflake.com/en/user-guide/tables-intro
- Query optimization
  https://docs.snowflake.com/en/user-guide/performance-query-optimization

論文

- The Snowflake Elastic Data Warehouse

関連技術

- Apache Iceberg

もしよければですが、ここまで理解すると Snowflakeの設計思想の90%くらいまで来ています。

ただ実はもう一つだけ 超重要なテーマがあります。

それは

「Snowflakeがなぜ“データレイクハウス”ではなく“DWH”なのか」

です。

これは

- Snowflake
- Databricks
- Iceberg

の思想の違いに直結します。

もし興味があれば、この3つのアーキテクチャの本質的な違いを図付きで解説します。
（データ基盤設計ではかなり重要です）

では最後に、Snowflake / Databricks / Iceberg のアーキテクチャ思想の違いを整理します。
これはデータ基盤設計では非常に重要で、現在の DWH vs Lakehouse 議論の中心でもあります。

説明の流れは次の通りです。

1. DWH と Data Lake の歴史
2. Snowflake の思想
3. Lakehouse（Databricks）の思想
4. Iceberg / Delta の役割
5. 実務でのアーキテクチャ選択

参考情報源も併記します。

1 DWHとデータレイクの歴史

もともとデータ基盤は次の2つに分かれていました。

種類	
目的	
DWH	BI分析
Data Lake	データ保存

典型構造

ETL

```
 |
 ▼
Data Warehouse
(Teradata / Oracle)
```

問題

```
高コスト
スケール限界
```

そこで登場したのが

```
Data Lake
```

構造

```
S3
 ├── parquet
 ├── json
 ├── csv
```

しかしData Lakeには問題がありました。

```
schema管理
ACID
query性能
```

参考
The Snowflake Elastic Data Warehouse

2 Snowflakeの思想（Cloud DWH）

Snowflakeは

```
Cloud Data Warehouse
```

として設計されました。

特徴

```
storage / compute separation
```

構造

```
Object Storage
 |
▼
Micro-partitions
 |
▼
Virtual Warehouse
```

Snowflakeは

```
structured analytics
```

に最適化されています。

特徴

- SQL
- BI
- analytics
- performance

参考
https://docs.snowflake.com/en/user-guide/intro-key-concepts

3 Databricksの思想（Lakehouse）

Databricksは

```
Data Lake + Warehouse
```

を統合する

```
Lakehouse
```

という概念を提案しました。

構造

```
S3
```

```
 |
▼
Delta Lake
 |
▼
Spark SQL
```

イメージ

```
Data Lake
+
ACID
+
SQL
```

これが

```
Lakehouse
```

です。

参考
Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores

4 Iceberg / Delta の役割

Lakehouseでは

テーブルフォーマット

が重要です。

例

- Delta Lake
- Iceberg
- Hudi

Iceberg構造

```
table
├── metadata.json
├── manifest list
├── manifest file
└── parquet files
```

つまり

```
data = parquet
metadata = table format
```

です。

参考
Apache Iceberg

5 Snowflake vs Lakehouse

アーキテクチャ比較

Snowflake		
Lakehouse		
Storage	proprietary	parquet
Metadata	internal	iceberg/delta
Engine	proprietary	spark
Format	closed	open

Snowflake

```
managed system
```

Lakehouse

```
open ecosystem
```

です。

6 実務での選択

現在の業界は

3つのパターンがあります。

① Snowflake中心

```
S3
 |
▼
Snowflake
 |
▼
BI
```

用途

- BI
- analytics
- SQL

② Lakehouse

```
S3
 |
▼
```

```
Iceberg / Delta
 │
 ▼
Spark
```

用途

- ML
- ETL
- data engineering

③ ハイブリッド

最近多いです。

```
S3
 │
 ├── Iceberg
 │
 ▼
Databricks
 │
 ▼
Snowflake
```

役割

システム	
役割	
Databricks	ETL / ML
Snowflake	BI

7 Snowflakeの強み

Snowflakeが強い理由

```
managed
performance
simplicity
```

つまり

```
SQL analytics
```

に最適です。

8 Lakehouseの強み

Lakehouseの強み

```
open ecosystem
ML
streaming
```

つまり

```
data platform
```

向きです。

まとめ

データ基盤の3つの思想

```
DWH
```

```
Data Lake
Lakehouse
```

整理すると

```
Snowflake = Cloud DWH
Databricks = Lakehouse
Iceberg = table format
```

です。

参考情報

Snowflake

https://docs.snowflake.com/en/user-guide/intro-key-concepts

Snowflake論文

The Snowflake Elastic Data Warehouse

Lakehouse論文

Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores

Iceberg

Apache Iceberg

もしよければですが、
あなたの質問レベルを見ると データ基盤アーキテクト視点の話がかなり向いていそうです。

次のテーマを解説すると理解が一気に深まります。

「Snowflakeがなぜ“SQLエンジン”として強いのか（内部実装）」

具体的には

- distributed query engine
- joinアルゴリズム
- broadcast join
- shuffle join
- Snowflake execution plan

このあたりです。
希望あれば続けます。

では続けて、**Snowflakeがなぜ「SQLエンジンとして強いのか」**を、内部の分散クエリエンジンの観点から解説します。
主に次の内容です。

1. Snowflakeの分散クエリエンジン
2. クエリ実行計画（Execution Plan）
3. Joinアルゴリズム
4. Broadcast Join と Shuffle Join
5. Snowflakeのパイプライン実行

参考資料として、公式ドキュメントや研究論文も併記します。

1 Snowflakeの分散クエリエンジン

Snowflakeは MPP (Massively Parallel Processing) 型のクエリエンジンです。

```
Query
 |
▼
Query Optimizer
 |
▼
Execution Plan
```

```
 │
 ▼
Distributed Execution
 │
 ├── Node1
 ├── Node2
 ├── Node3
 └── Node4
```

Virtual Warehouseは内部的に

```
cluster
 ├── compute node
 ├── compute node
 ├── compute node
```

で構成されています。

各ノードは

- マイクロパーティションをスキャン
- ローカル処理
- 結果を集約

します。

参考
Snowflake Warehouses
https://docs.snowflake.com/en/user-guide/warehouses-overview

2 クエリ実行計画

SQLを実行すると

```
SQL
 ↓
Parser
 ↓
```

```
Logical Plan
↓
Physical Plan
↓
Execution
```

になります。

例

```
SELECT *
FROM orders o
JOIN customers c
ON o.customer_id = c.id
```

Optimizerは

```
join order
scan strategy
partition pruning
```

を決定します。

参考
Snowflake Query Optimization
https://docs.snowflake.com/en/user-guide/performance-query-optimization

3 SnowflakeのJoinアルゴリズム

Snowflakeでは主に3種類のJoinがあります。

Join	
特徴	
Hash Join	一般的
Broadcast Join	小さいテーブル
Merge Join	ソート済み

Hash Join

一般的なJoinです。

```
Table A
Table B
```

処理

```
hash table build
↓
probe
```

イメージ

```
Build side
└── hash table

Probe side
└── scan
```

4 Broadcast Join

小さいテーブルの場合

small table

を

全ノードにコピー

します。

```
Node1 ──┐
Node2 ──┼── small table copy
Node3 ──┘
```

その後

local join

します。

メリット

shuffle不要

高速です。

5 Shuffle Join

大きいテーブル同士の場合

partition redistribution

が必要です。

```
Table A partition
Table B partition
```

を

hash(key)

で分配します。

イメージ

```
Node1 ──┐
Node2 ──┼── shuffle
Node3 ──┘
```

その後

```
local join
```

します。

6 Snowflakeのパイプライン実行

Snowflakeは

```
pipeline execution
```

を使います。

つまり

```
scan
join
aggregation
```

を

```
同時並行
```

で実行します。

イメージ

```
scan partitions
↓
partial join
↓
partial aggregation
```

結果

```
memory usage ↓
```

```
latency ↓
```

になります。

参考
The Snowflake Elastic Data Warehouse

7 Snowflake Execution Pipeline

実際の処理

```
partition scan
 |
▼
filter
 |
▼
join
 |
▼
aggregate
 |
▼
result
```

各処理は

```
distributed operator
```

です。

8 Result Aggregation

最終結果は

```
partial results
```

を

```
final node
```

が集約します。

```
node1 partial
node2 partial
node3 partial
```

↓

```
final result
```

9 Snowflakeが速い理由（SQLエンジン）

SnowflakeのSQL性能は

次の3つです。

```
1 partition pruning
2 distributed execution
3 pipeline processing
```

つまり

```
scan最小化
+
並列処理
```

です。

まとめ

Snowflake SQLエンジン

```
SQL
 │
 ▼
Optimizer
 │
 ▼
Distributed Execution
 │
 ├── scan partitions
 ├── join
 ├── aggregate
```

実行モデル

```
MPP
+
pipeline
```

です。

主な情報源

Snowflake公式

- Warehouses
  https://docs.snowflake.com/en/user-guide/warehouses-overview
- Query Optimization
  https://docs.snowflake.com/en/user-guide/performance-query-optimization
- Query Processing
  https://docs.snowflake.com/en/user-guide/querying-overview

研究論文

- The Snowflake Elastic Data Warehouse

もしよければですが、ここまで理解すると Snowflakeアーキテクチャのほぼ全体です。

ただ、実務ではさらに重要なテーマがあります。

それは

「Snowflakeで遅いクエリが発生する本当の理由」

です。

実際にはほとんどの場合

- partition pruningが効いていない
- join戦略ミス
- clustering崩壊

が原因です。

希望あれば Snowflakeパフォーマンスチューニング完全版を解説します。
（実務で一番役に立つ内容です）

では Snowflake パフォーマンスチューニング完全版をまとめます。
実務で Snowflake の遅いクエリの原因の 90%はこの5つです。

1. Partition pruning
2. クラスタリング設計
3. Join戦略
4. ファイルサイズ / ロード方法
5. Warehouse設計

Snowflake公式資料や論文を根拠に整理します。

1 Partition Pruning（最重要）

Snowflakeの性能は基本的に

```
Scan量
```

で決まります。

Snowflakeは

```
micro-partition metadata
```

を使って

```
不要パーティションを読み飛ばす
```

仕組みです。

イメージ

```
partition1 2024-01-01～2024-01-05
partition2 2024-01-06～2024-01-10
partition3 2024-01-11～2024-01-15
```

クエリ

```
SELECT *
FROM orders
WHERE order_date = '2024-01-07'
```

結果

```
partition2のみ読む
```

これが

```
Micro-partition pruning
```

です。

参考
Snowflake Micro-partitions
https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

2 Pruningが効かない典型例

NG① 関数

```
WHERE YEAR(order_date) = 2024
```

これは

```
metadata min/max
```

が使えません。

良い例

```
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31'
```

NG② LIKE

```
WHERE email LIKE '%gmail.com'
```

prefix検索以外は

```
full scan
```

になります。

NG③ CAST

```
WHERE CAST(order_date AS STRING)
```

これも pruning 不可。

3 クラスタリング設計

クラスタリングとは

```
似た値を同じmicro-partitionに置く
```

ことです。

悪い例

```
partition1
2024-01
2024-07
2024-03
```

良い例

```
partition1 2024-01-01～2024-01-10
partition2 2024-01-11～2024-01-20
```

クラスタリングキー

```
CLUSTER BY(order_date)
```

参考
Snowflake Clustering Keys
https://docs.snowflake.com/en/user-guide/tables-clustering-keys

4 Join戦略チューニング

Snowflake Joinは主に

Join	
用途	
Broadcast join	小さいテーブル
Shuffle join	大きいテーブル

Broadcast join

```
small table
↓
全ノードコピー
```

高速です。

例

```
fact_orders
join
dim_customer
```

dim_customerが小さい場合

```
broadcast join
```

になります。

参考
Snowflake Query Optimization
https://docs.snowflake.com/en/user-guide/performance-query-optimization

5 Join順序

Optimizerは

```
small table first
```

を選びます。

しかし統計が悪いと

```
join explosion
```

が起きます。

例

```
fact_orders
join
fact_events
```

これはかなり重いです。

6 ファイルサイズ

Snowflakeでは

```
COPY INTO
```

のファイルサイズが重要です。

理想

```
100MB 前後
```

悪い例

```
1MB files
```

これを

```
small files problem
```

と言います。

理由

```
micro-partition爆増
metadata増加
```

参考
Snowflake Data Loading Best Practices

https://docs.snowflake.com/en/user-guide/data-load-considerations

7 MERGEの落とし穴

Snowflakeは

```
copy-on-write
```

です。

つまり

```
UPDATE
MERGE
```

は

```
新micro-partition作成
```

になります。

例

```
10TB table
MERGE
```

すると

```
大量パーティション再生成
```

になります。

8 Warehouseチューニング

Warehouseサイズ

Size	
Nodes	
XS	1
S	2
M	4
L	8

Warehouseを大きくすると

`parallel scan`

が増えます。

しかし

`scan量が大きい`

と

`warehouseを大きくしても遅い`

です。

9 Result Cache

Snowflakeは

`result cache`

を持っています。

同じクエリなら

`0秒`

で返ります。

条件

```
query identical
data unchanged
```

参考
Snowflake Result Cache
https://docs.snowflake.com/en/user-guide/querying-persisted-results

10 Query Profileの使い方

Snowflakeの調査は

```
Query Profile
```

を見ます。

見るポイント

```
scan bytes
partitions scanned
join type
```

例

```
Table Scan
Partitions scanned: 12000
```

これは

```
pruning failure
```

です。

11 Snowflakeチューニング優先順位

Snowflakeエンジニアの定石

① Partition pruning
② Clustering
③ Join戦略
④ File size
⑤ Warehouse

です。

つまり

```
scan量削減
```

が最重要です。

まとめ

Snowflakeチューニングの本質

```
scan data
↓
reduce scan
↓
parallelize scan
```

です。

主な情報源

Snowflake公式

Architecture
https://docs.snowflake.com/en/user-guide/intro-key-concepts

Micro-partitions
https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

Query optimization
https://docs.snowflake.com/en/user-guide/performance-query-optimization

Clustering
https://docs.snowflake.com/en/user-guide/tables-clustering-keys

Data loading
https://docs.snowflake.com/en/user-guide/data-load-considerations

Result cache
https://docs.snowflake.com/en/user-guide/querying-persisted-results

研究論文

- The Snowflake Elastic Data Warehouse

もしよければですが、ここまで理解すると Snowflakeアーキテクトレベルの知識です。

次に解説するとさらに理解が深まるのが

「Snowflake内部のメタデータ管理アーキテクチャ」

です。

これは普通のエンジニアは知らない内容で

- catalog設計
- transaction管理
- snapshot isolation
- metadata scaling

に関係します。

かなり面白いテーマなので、希望あれば解説します。

では最後に、Snowflake内部のメタデータ管理アーキテクチャを解説します。
これはSnowflakeのスケーラビリティやトランザクションの仕組みを理解するうえで重要です。

内容は次の流れで説明します。

1. Snowflakeのメタデータとは
2. メタデータサービスの役割
3. スナップショット分離（Snapshot Isolation）
4. Time Travel と Fail-safe
5. メタデータスケーリング

主にSnowflake公式資料と論文を基にまとめています。

1 Snowflakeのメタデータとは

Snowflakeでは、データそのものはクラウドストレージに置かれますが、
データを管理する情報（メタデータ）はCloud Services層が管理します。

```
Snowflake Architecture

Client
 │
▼
Cloud Services
 ├── Metadata
 ├── Query optimizer
 ├── Transaction manager
 │
▼
Compute (Virtual Warehouse)
```

```
 |
▼
Storage (Object Storage)
```

メタデータの例

- テーブルスキーマ
- マイクロパーティション情報
- クエリ履歴
- アクセス制御
- 統計情報

参考
Snowflake Architecture
https://docs.snowflake.com/en/user-guide/intro-key-concepts

2 メタデータサービスの役割

SnowflakeのCloud Services層には

```
Metadata Services
```

があります。

主な役割

```
table catalog
schema management
query planning
access control
transaction management
```

イメージ

```
Metadata Catalog
 ├── database
 │  ├── schema
 │  │  ├── table
```

```
│ │ │ ├── micro-partitions
│ │ │ └── statistics
```

このカタログ情報があるため

- クエリ計画
- パーティションプルーニング
- トランザクション管理

が可能になります。

参考
Snowflake Metadata
https://docs.snowflake.com/en/user-guide/intro-key-concepts

3 Snapshot Isolation（スナップショット分離）

Snowflakeのトランザクションは

```
Snapshot Isolation
```

で動作します。

概念

```
transaction start
↓
snapshot created
↓
query sees consistent data
```

例

ユーザーA

```
SELECT * FROM orders;
```

ユーザーB

```
UPDATE orders SET price = 100;
```

この場合

```
User A → 旧データ
User B → 新データ
```

をそれぞれ見ます。

これは

```
copy-on-write
```

方式で実現されています。

参考
Snowflake Transactions
https://docs.snowflake.com/en/user-guide/transactions

4 Time Travel

Snowflakeの特徴的な機能の一つです。

```
SELECT *
FROM orders
AT (TIMESTAMP => '2026-03-10 10:00');
```

内部では

```
old micro-partition
new micro-partition
```

が保存されています。

Time Travel期間

エディション	
保持	
Standard	1日
Enterprise	最大90日

参考
Snowflake Time Travel
https://docs.snowflake.com/en/user-guide/data-time-travel

5 Fail-safe

Time Travelの後に

Fail-safe

という期間があります。

目的

災害復旧

です。

期間

7日

この間は

Snowflakeサポートのみ復旧可能

です。

参考
Snowflake Fail-safe
https://docs.snowflake.com/en/user-guide/data-failsafe

6 メタデータスケーリング

Snowflakeは大量のテーブル・パーティションを管理します。

例

```
10TB table
↓
50000+ micro-partitions
```

そのため

```
metadata distributed architecture
```

を採用しています。

イメージ

```
Metadata Service
 ├── partition catalog
 ├── statistics store
 ├── query history
```

これにより

```
millions of partitions
```

を管理できます。

論文でも説明されています。

参考
The Snowflake Elastic Data Warehouse

7 Snowflakeメタデータの重要性

Snowflakeでは

```
metadata = performance
```

です。

理由

```
partition pruning
query planning
statistics
```

すべてメタデータに依存します。

まとめ

Snowflakeの内部管理構造

```
Object Storage
 |
▼
Micro-partitions
 |
▼
Metadata Catalog
 |
▼
Cloud Services
```

そして

```
Snapshot isolation
+
copy-on-write
```

により

- Time Travel
- 高並行トランザクション
- 分散クエリ

が実現されています。

主な情報源

Snowflake公式

Architecture
https://docs.snowflake.com/en/user-guide/intro-key-concepts

Transactions
https://docs.snowflake.com/en/user-guide/transactions

Time Travel
https://docs.snowflake.com/en/user-guide/data-time-travel

Fail-safe
https://docs.snowflake.com/en/user-guide/data-failsafe

研究論文

- The Snowflake Elastic Data Warehouse

もし興味があれば、次に

「Snowflake内部のクエリオプティマイザ」

を解説できます。
これは実際のデータ基盤設計でかなり重要で、

- 統計情報
- join reorder

- cost model

などが関係します。

はい。結論から言うと、Snowflakeを「業務アプリのデータストア」として使うなら、まずそのアプリが OLTP 型なのか、分析寄りなのかを切り分けるのが最重要です。
Snowflake の通常テーブルは分析基盤向けで、近年は Hybrid Tables により低遅延・高スループットのランダム read/write、行ロック、主キー/外部キー制約といったトランザクション寄りの要件にも対応しています。ただし、Hybrid Tables には機能差やクォータがあり、一般的なRDBをそのまま置き換える前提では評価が必要です。

考慮点は、実務では次の順番で整理すると失敗しにくいです。

1. まず「どの種類の処理」を載せるのか

- 集計、検索、レポート、分析API、ダッシュボードが中心なら、Snowflakeはかなり相性がいいです。
- 一方で、高頻度な単票更新、厳しいミリ秒応答、複雑なトランザクション制御、強いOLTP要件が主役なら、通常テーブル前提では厳しく、Hybrid Tables 前提での検証が必要です。Hybrid Tables はトランザクション用途向けですが、標準テーブルとは別物として設計を考えるべきです。

2. テーブル種別を最初に決める

Snowflake には少なくとも次の考え分けがあります。

- 標準テーブル: 分析・ELT・集計向け
- Hybrid Tables: 低遅延の point read/write と高並行な業務処理向け
- Interactive tables / interactive warehouses: 高並行・低遅延の対話型クエリ向け。ただし利用可能リージョンやSQL上の制約を確認すべきです。

なので、アプリの中でも

- 台帳・注文・ステータス更新 → Hybrid Tables 候補
- 参照系・分析系 → 標準テーブルや interactive 系候補
  のように、用途ごとに使い分ける設計が自然です。

3. レイテンシ要件を甘く見ない

Snowflakeは「速い」ですが、何に対して速いかが重要です。
Hybrid Tables ですら、まずは実際のドライバ経由の負荷試験で確認する必要があります。Snowflake 公式でも、Snowsight 上の表示値は driver-based workload の性能評価には向かないと明記しています。つまり、画面で速く見えても、業務アプリの本番SDK/ドライバ経由では別です。

4. 同時実行数と更新パターンを確認する

Hybrid Tables には既定で、1データベースあたり約8,000 ops/sec の balanced workload 目安や、ハイブリッドストレージ 2TB/DB などのクォータがあります。大量同時更新や急激なスパイクがある業務では、この制約が設計判断に直結します。

このため、次を事前に数値化した方がいいです。

- 秒間 read / write 件数
- 同時接続数
- ピーク時間帯
- 1件更新か、まとめ更新か
- 参照:更新の比率
  これが曖昧なまま選ぶと、後で「使えなくはないが苦しい」状態になりやすいです。これは上記クォータと Snowflake の推奨評価手順からの実務的な帰結です。

5. 制約・SQL互換性・機能差を確認する

Hybrid Tables は魅力的ですが、制限事項や未対応機能があります。Snowflake 自身が limitations ページを分けているので、採用前提なら必読です。特に、既存アプリが PostgreSQL / MySQL / Oracle 前提で書かれていると、SQLや機能の差分確認が必要です。

6. コストモデルを理解する

Snowflakeは一般的なDBのように「常時1台」ではなく、ストレージとコンピュートを分離して考えます。Hybrid Tables には専用のストレージ課金もあり、クエリや処理のためのコンピュートも別で考える必要があります。低遅延要件を満たすために常時稼働寄りにすると、想像よりコストが伸びることがあります。

特に業務アプリ用途では、

- 24時間アクセスがあるか
- 夜間停止できるか
- バッチ中心か常時オンラインか
  でコスト感が大きく変わります。これは Snowflake の分離アーキテクチャからくる本質です。

7. セキュリティ/権限制御を最初に整理する

Snowflakeはロールベースの権限管理が中心です。テーブル、ビュー、スキーマ、DB、ウェアハウスごとに権限を整理し、アプリ用ロールと管理用ロールを分けるのが基本です。Hybrid Tables も権限付与は通常の TABLE として扱います。

実務では、

- アプリ実行ロール
- ETL/バッチロール
- 開発者参照ロール
- 運用管理ロール
  を分離し、本番データへの直接更新権限を絞る設計が重要です。これは Snowflake の権限モデルに沿った王道です。

8. 運用監視をどうするか

Hybrid Tables では、SHOW TRANSACTIONS、SHOW LOCKS、LOCK WAIT HISTORY などでトランザクションやロック待ちを監視できます。つまり、アプリ用データストアとして使うなら、単にSQLを打つだけでなく、ロック競合や待ち時間を見る運用設計が必要です。

9.「Snowflakeだけで全部やる」かを見極める

実務では、全部Snowflakeに寄せるより、次のような分離がよく合います。

- SoR / 厳密OLTP は別DB
- 分析・統合・配信・参照強化 は Snowflake
- もしくは 更新系の一部だけ Hybrid Tables
  Snowflake 公式も、Hybrid Tables を「トランザクションと分析を一つのプラットフォームで扱う」ための選択肢として位置づけていますが、制約つきなので、全面置換より適材適所の混在設計が安全です。

あなたが最初に決めるべきチェック項目

これを埋めると判断しやすいです。

- 1リクエストの許容応答時間は何msか
- 1秒あたりの read/write 件数はどれくらいか
- 更新は単一行中心か、複数表トランザクションか
- 業務の正本DBにしたいのか、参照/統合ストアにしたいのか
- データ量はどれくらいか
- 24時間オンラインか、営業時間中心か
- 監査/権限制御/暗号化の要求はどの程度か
- 既存アプリはどのDB前提で作られているか
  これらは、Hybrid Tables の性能特性・制約・権限・コスト構造に直結します。

かなり実務的な結論

Snowflakeを「業務アプリのデータストア」にすること自体は可能ですが、

- 分析・参照・統合基盤として使うのか

- 更新系の本番DBとして使うのか
  で難易度が大きく違います。

更新系の本番DBとして使うなら、Hybrid Tables 前提で、性能試験・制限確認・コスト試算・ロック監視設計までやってから決めるのが安全です。
一方、参照系や分析系が中心なら、Snowflakeはかなり強い候補です。

あなたの想定が
「業務アプリの正本DB」なのか、
「分析も兼ねた参照用ストア」なのかで、設計の勘所がかなり変わります。
その前提に合わせて、次は メリデメを業務アプリ目線で具体化 して整理できます。

Snowflake へのデータ登録（取り込み）は、実務では 大きく6パターンに分かれます。
用途（リアルタイム / バッチ / 外部連携 / データレイク）で選びます。

1. COPY INTO（最も一般的：バッチ）

一番よく使われる方法です。
S3 / Azure Blob / GCS に置いたファイルを Snowflake にロードします。

```
S3 → Snowflake
```

```
COPY INTO my_table
FROM @my_stage/data/
FILE_FORMAT = (TYPE = CSV);
```

特徴

- 大量データに強い
- Snowflake公式の王道
- データレイク連携と相性良い

よくある構成

```
外部システム
↓
S3
↓
Snowflake Stage
↓
COPY INTO
↓
Table
```

AWSデータレイク構成ではほぼこの方式です。

2. Snowpipe（自動取り込み）

ファイル到着をトリガーに自動ロードします。

```
S3 → Snowpipe → Snowflake
```

構成

```
S3
↓（イベント通知）
SNS / SQS
↓
Snowpipe
↓
Table
```

特徴

- 自動ロード
- ほぼリアルタイム
- バッチ不要

典型用途

- データレイク
- ログ収集
- IoT
- 外部システムファイル連携

3. Snowpipe Streaming（リアルタイム）

Snowflakeの最新のリアルタイム取り込み機能

```
アプリ → Snowpipe Streaming → Snowflake
```

特徴

- 秒単位でデータ反映
- Kafka不要
- APIベース

典型用途

- イベントデータ

- トレードログ
- センサー
- アプリログ

4. INSERT / MERGE（アプリ連携）

普通のSQLでデータ登録します。

```
INSERT INTO table VALUES (1,'abc');
```

または

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN UPDATE
WHEN NOT MATCHED THEN INSERT
```

特徴

- RDBと同じ
- OLTP的
- 少量データ向き

注意

Snowflakeは OLTP用途には最適ではない
（Hybrid Tablesを除く）

5. コネクタ / ETLツール

外部ツールから直接ロードする方法。

例

- Fivetran
- Airbyte
- dbt
- Informatica
- Matillion
- Talend

典型構成

```
RDB
↓
ETL
↓
Snowflake
```

最近は ELT（Snowflake側で変換） が主流。

6. Kafka / Streaming連携

ストリームデータの取り込み。

```
Kafka → Snowflake Connector
```

または

```
Kafka → Snowpipe Streaming
```

特徴

リアルタイムデータ基盤

実務で一番多い構成

ほとんどの企業はこの形です。

```
業務システム
↓
データレイク(S3)
↓
Snowpipe / COPY INTO
↓
Snowflake
↓
BI / 分析
```

理由

- データレイクを中心にできる
- 再処理可能
- 安全
- スケールする

AWS × Snowflake の典型アーキテクチャ

```
外部システム
↓
S3
↓
EventBridge
↓
Snowpipe
↓
Snowflake
↓
BI / アプリ
```

この構成が モダンデータアーキテクチャ です。

取り込み方法の選び方

方法	向いている用途
COPY INTO	大量バッチ
Snowpipe	自動ファイル連携
Snowpipe Streaming	リアルタイム
INSERT	少量更新
ETLツール	SaaS / DB連携
Kafka	ストリーム

Snowflake設計で重要なポイント

Snowflakeでは

データ登録方法 = アーキテクチャ

になります。

最初に決めるべきこと

- リアルタイムか
- バッチか
- データレイク併用か
- CDCが必要か

もしよければ教えてください。
かなり重要なので聞きたいです。

1. Snowflakeは
    - データレイクの分析基盤？
    - 業務アプリのDB？
2.
3. データの元
    - RDB
    - 外部API
    - ファイル
    - SaaS
4.
5. 更新頻度
    - リアルタイム
    - 数分
    - 日次
6.

これが分かると、AWS + Snowflake の最適アーキテクチャをかなり具体的に提案できます。

（実務でよくある構成を図付きで説明できます。）